CS 524 – Introduction to Cloud Computing

Final Project

NAME: Aisiri Mandya Rajashekar

E-mail: arajashe@stevens.edu

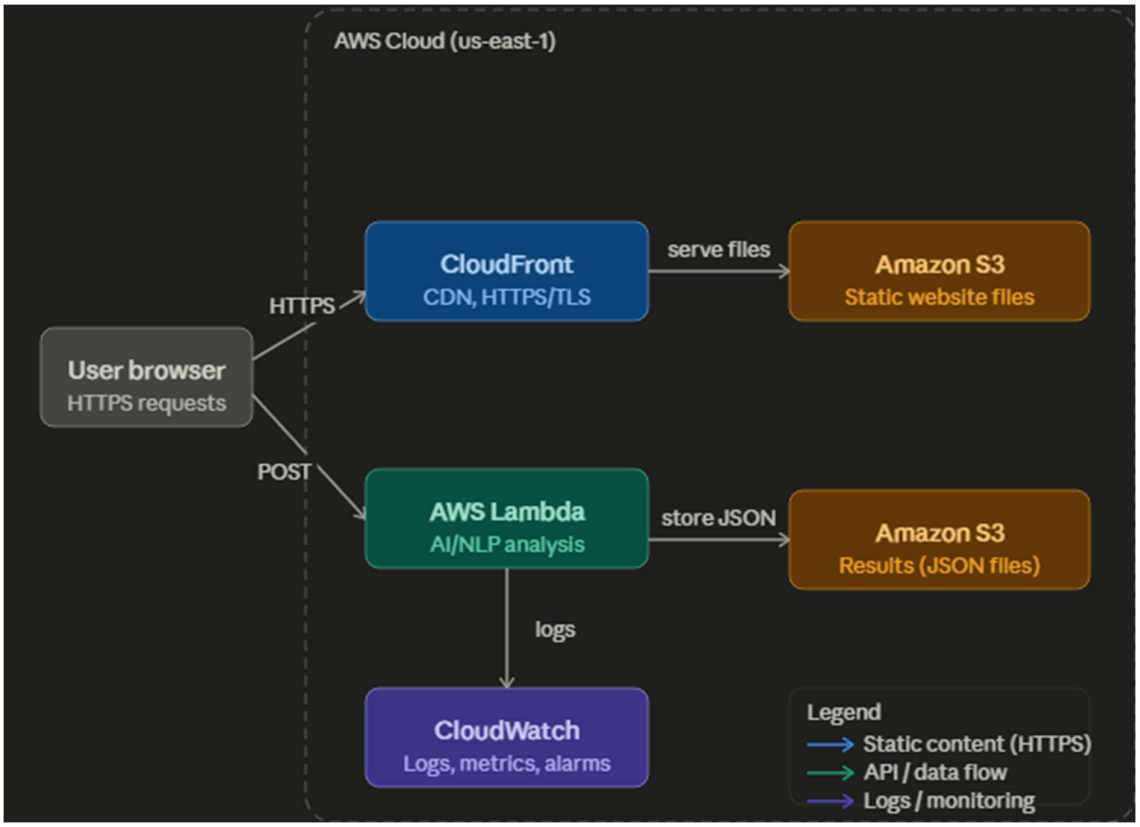
CWID: 20043750

AI-POWERED SECURE WEB APPLICATION

Building an Intelligent Static Website on AWS

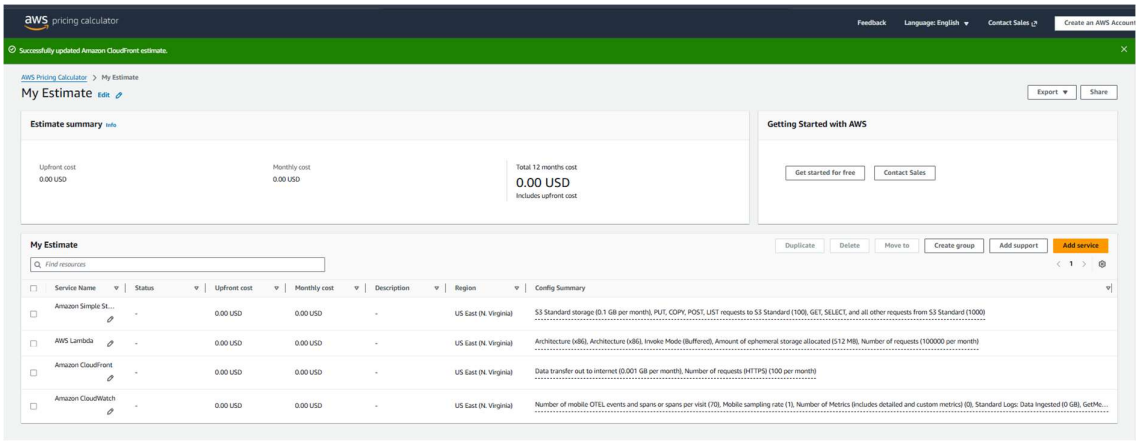
Phase 1- Architecture Diagram & Cost Estimate

Step 1: Architecture diagram



This setup lets users access a website through CloudFront, send their input to a Lambda function for processing, and store the results in S3 while tracking activity in CloudWatch.

Step 2: AWS Pricing Calculator



I opened the AWS Pricing Calculator and added S3, Lambda, CloudFront, and CloudWatch with low usage values. The total came out to $0.00 which confirmed everything fits within the free tier.

Step 3: Design Explanation

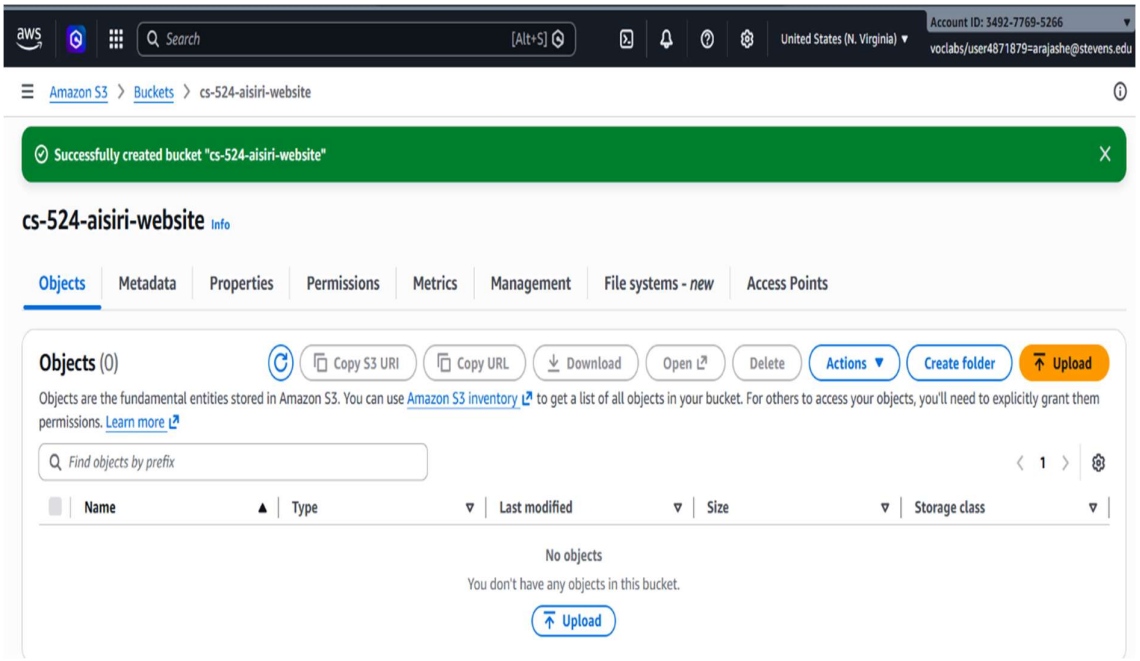
This architecture was designed to be secure, scalable, and cost-effective using managed AWS services. The static frontend (HTML, CSS, JavaScript) is hosted on Amazon S3 and delivered globally through CloudFront, which enforces HTTPS/TLS encryption for all client connections. This separation of frontend and backend allows the website to scale effortlessly without any server management.

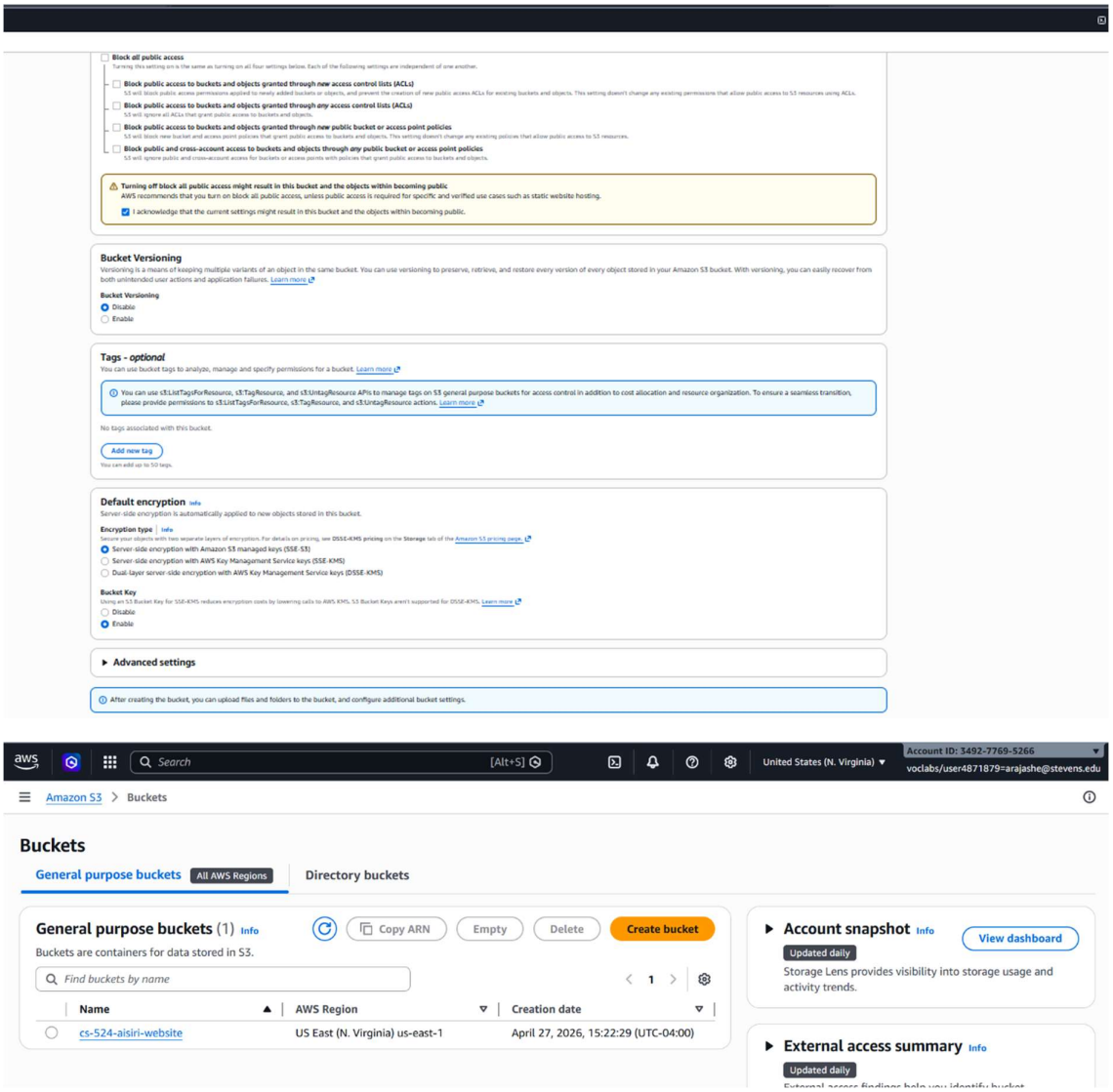
When a user submits text for analysis, the browser sends a POST request directly to a Lambda Function URL. AWS Lambda runs the AI/NLP processing in a serverless environment, meaning it only consumes resources when invoked. Analysis results are stored as JSON files back in S3, creating a persistent record of each request. CloudWatch automatically captures Lambda execution logs, enabling monitoring, dashboards, and alarm-based notifications.

This architecture avoids the need for EC2 instances or traditional servers entirely, keeping costs within the AWS free tier for the expected workload while maintaining strong security through SSE-S3 encryption and HTTPS enforcement at every layer.

Phase 2 — Create S3 Bucket & Host Static Website

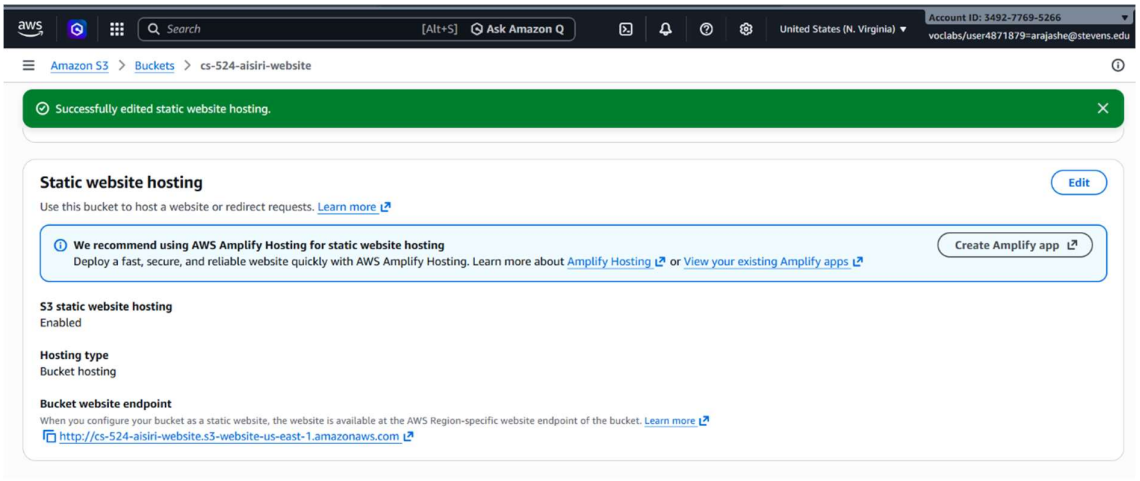
Step 1: Create the S3 Bucket





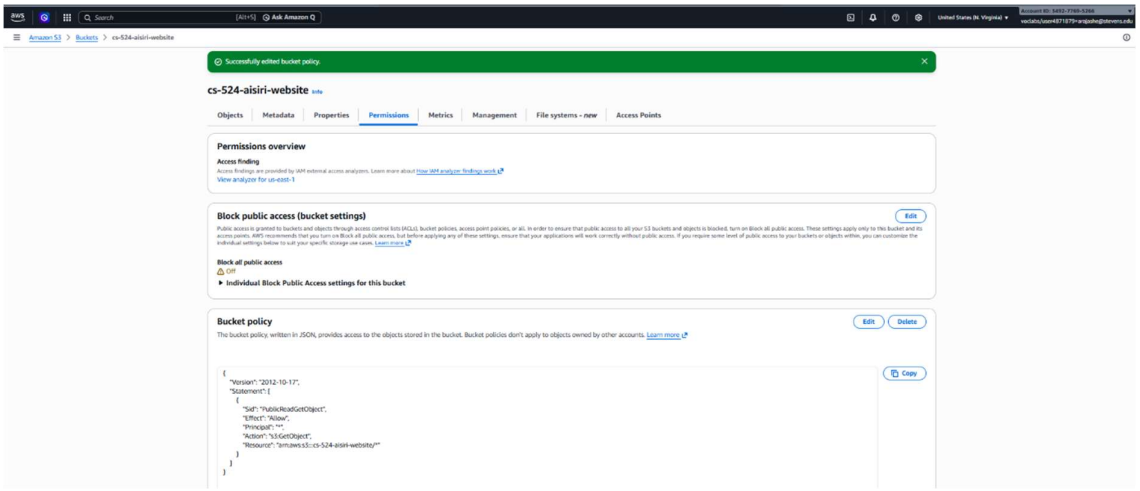
I created a new S3 bucket called cs-524-aisiri-website in the us-east-1 region. I unchecked Block all public access and acknowledged the warning before creating it.

Phase 2: Static website hosting turned on, showing the bucket endpoint URL



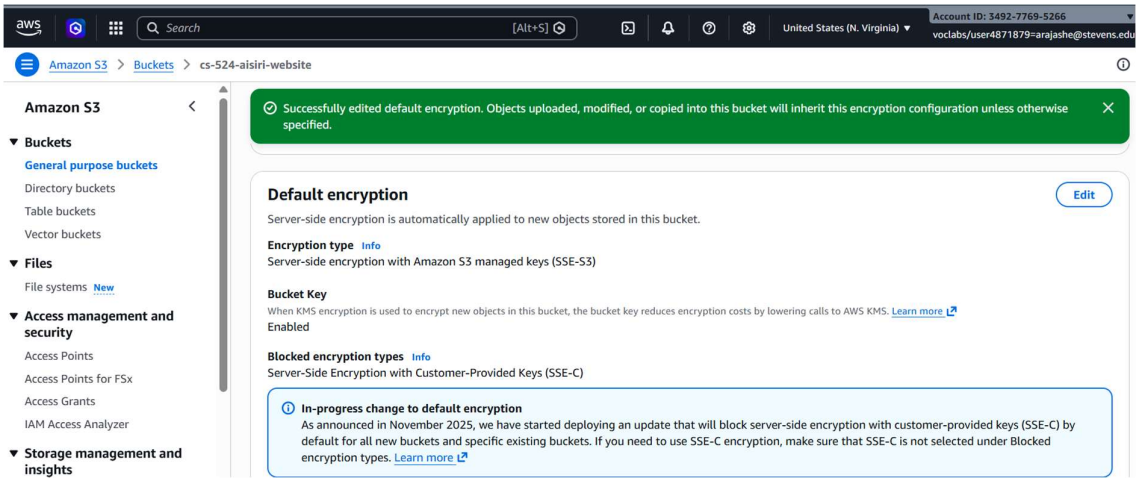
I went to the Properties tab and enabled static website hosting with index.html as the index document. The bucket endpoint URL appeared at the bottom which I saved for later.

Step 3: Bucket policy added to allow public read access



I pasted the public read policy into the bucket policy editor under the Permissions tab. After saving, the bucket was accessible publicly for static hosting.

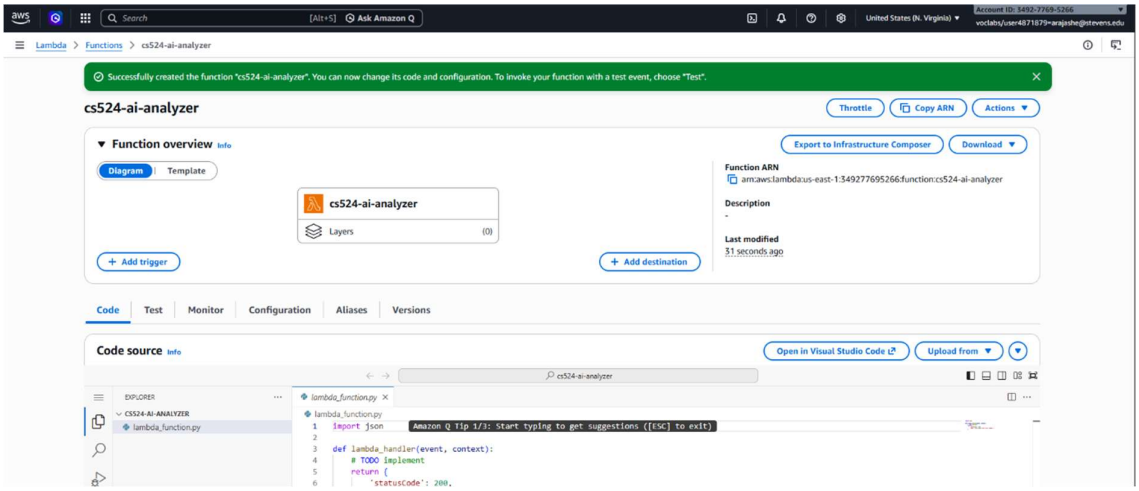
Step 4: Default encryption set to SSE-S3



I went to Properties, found Default encryption, and changed it to SSE-S3. This makes sure any file stored in the bucket is encrypted automatically.

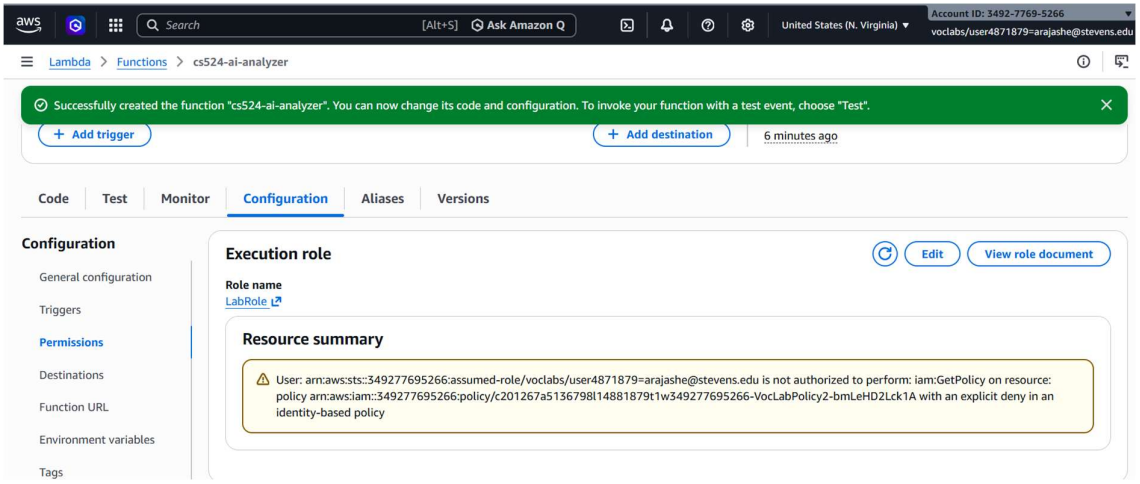
Phase 3: Create Lambda Function (AI Backend)

Step 1a: Lambda function cs524-ai-analyzer successfully created



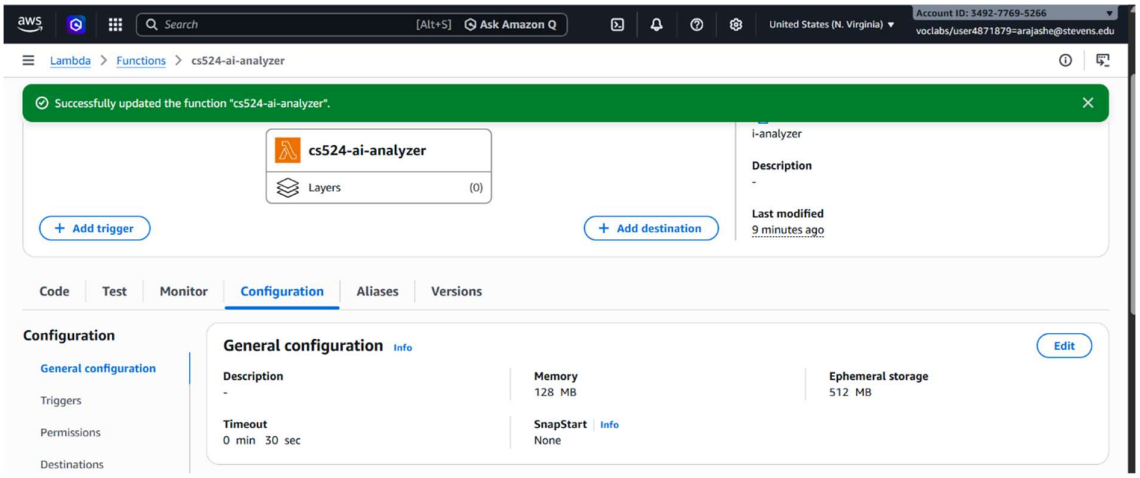
I created a new Lambda function from scratch using Python 3.12 and x86_64 architecture. The green success banner confirmed it was created without any issues.

Step 1b: Permissions tab showing LabRole is attached as the execution role



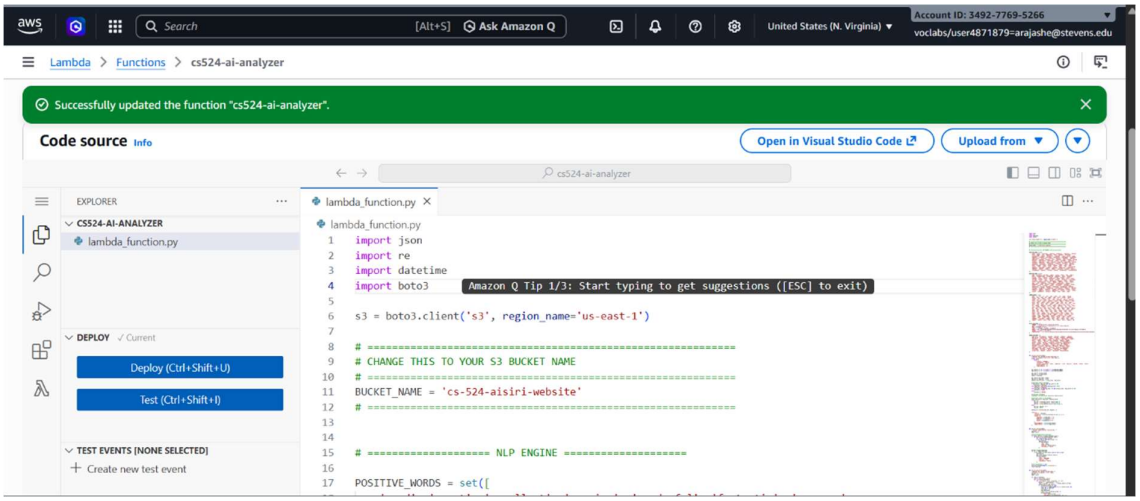
I went to the Configuration tab and clicked Permissions to verify the execution role. It showed LabRole was correctly attached as required.

Step 2: Timeout updated to 30 seconds under general configuration



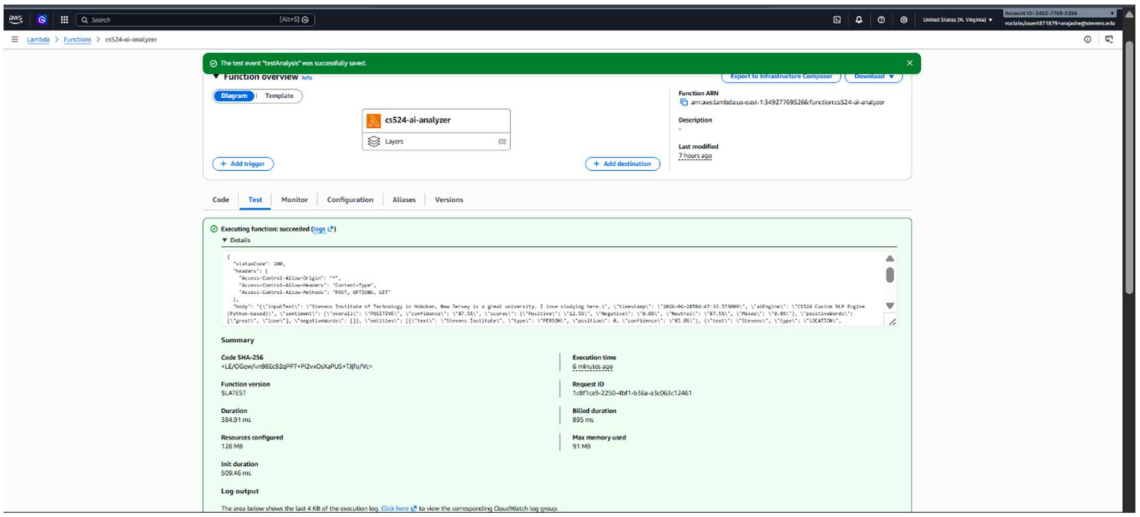
I went to Configuration, then General configuration, and clicked Edit. I changed the timeout from 3 seconds to 30 seconds and saved.

Step 3: CA's Lambda code pasted and deployed successfully



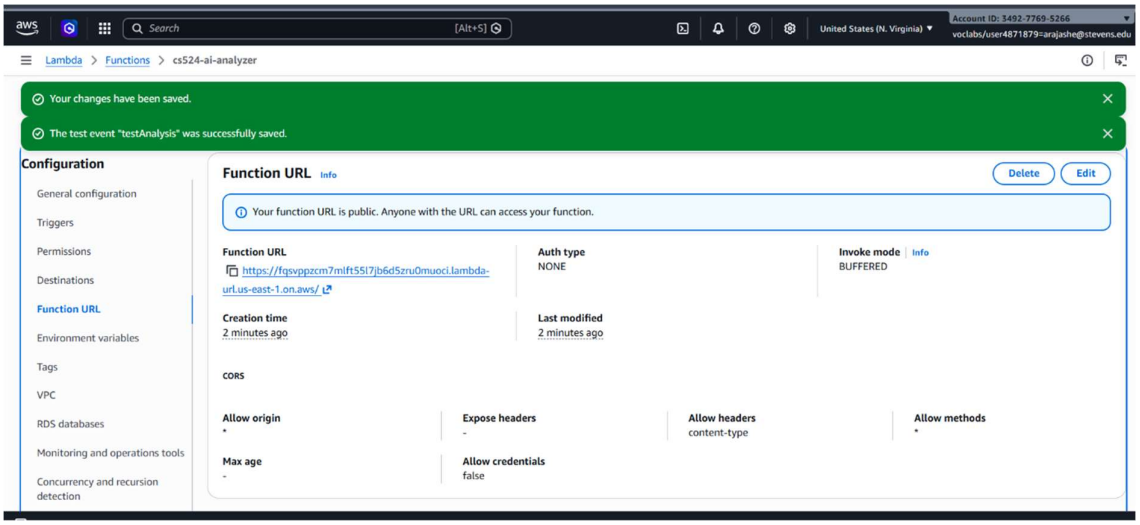
I pasted the CA's Lambda code into the editor and updated the BUCKET_NAME variable to match my S3 bucket. After clicking Deploy the green bar showed it was successfully updated.

Step 4: Test result showing the function ran successfully with statusCode 200



I created a test event called testAnalysis with the sample JSON provided in the announcement. Running it returned statusCode 200 with full analysis results showing it was working.

Step 5: Function URL created with CORS settings showing allow origin set to star



I went to Configuration then Function URL and created a new URL with Auth type NONE. I enabled CORS with allow origin set to star and allow headers set to content-type.

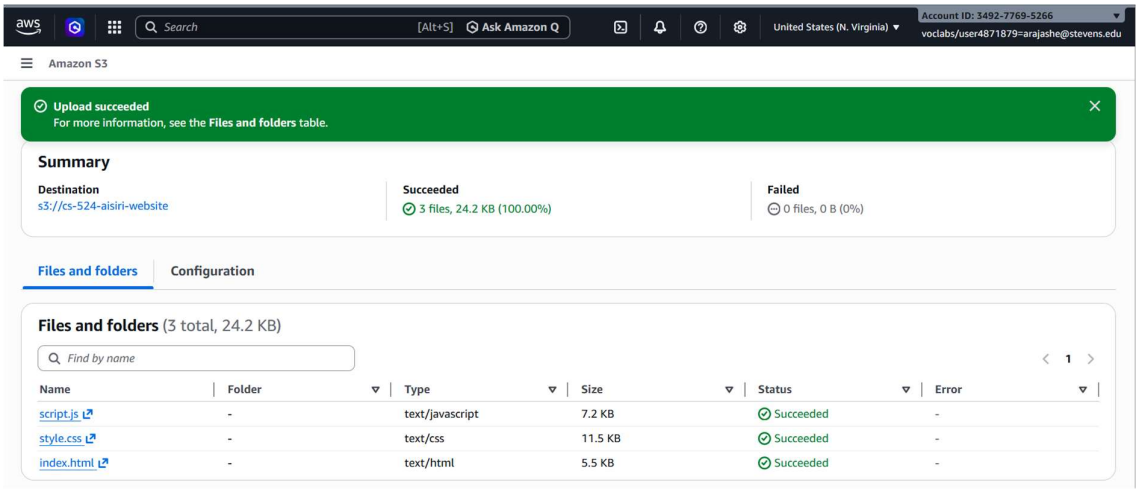
Phase 4: Upload Website & Connect to Lambda

Step 1: script.js updated with the Lambda Function URL before uploading

```
// ============================================================
// CS524 - AI Text Analyzer
// Lambda Function URL - update this with your actual URL
// ============================================================
const API_URL = 'https://fqsvppzcm7mlft55l7jb6d5zru0muoci.lambda-url.us-east-1.on.aws/';

// ===== Character Counter =====
const textInput = document.getElementById('textInput');
const charCount = document.getElementById('charCount');

textInput.addEventListener('input', () => {
  const len = textInput.value.length;
  charCount.textContent = `${len} character${len !== 1 ? 's' : ''}`;
});

// ===== Allow Ctrl+Enter to submit =====
textInput.addEventListener('keydown', (e) => {
  if (e.key === 'Enter' && e.ctrlKey) analyzeText();
});

// ===== Main Analyze Function =====
```

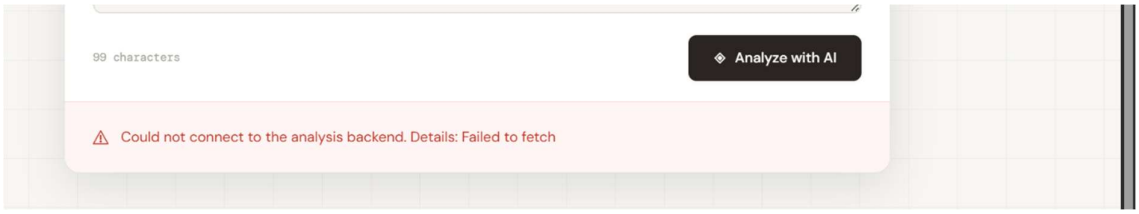
I opened script.js on my computer and replaced the placeholder text with my actual Lambda Function URL. I saved the file before uploading anything to S3.

Step 2: All three files uploaded to the S3 bucket

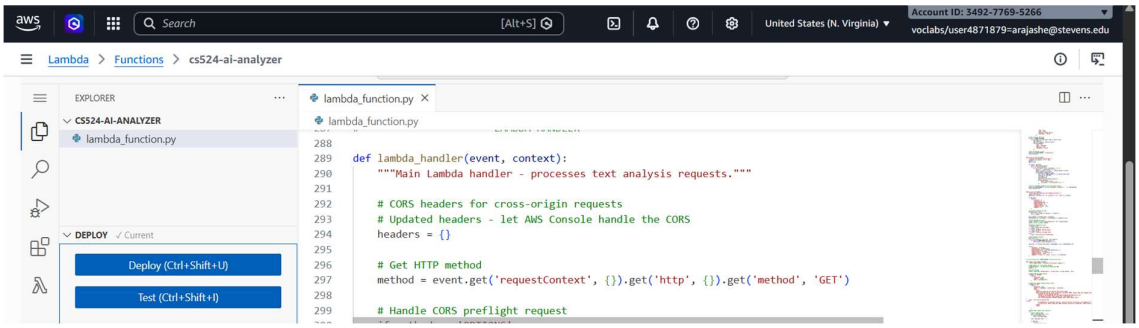


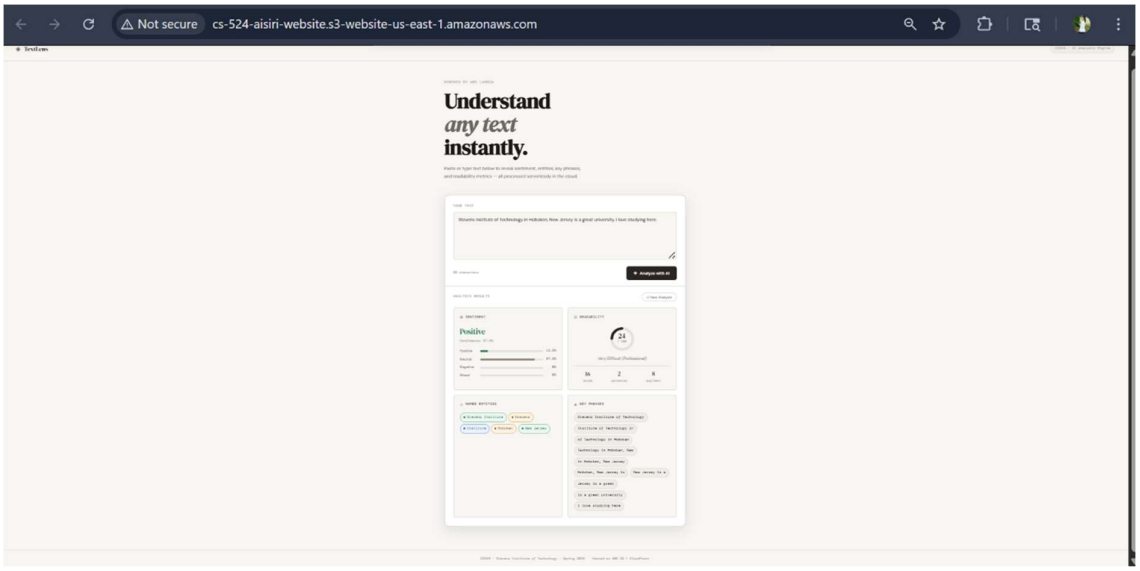
I went to my S3 bucket and uploaded all three files — index.html, style.css, and script.js. All three showed up in the bucket after the upload completed.

Step 3: Website loading through the S3 endpoint URL



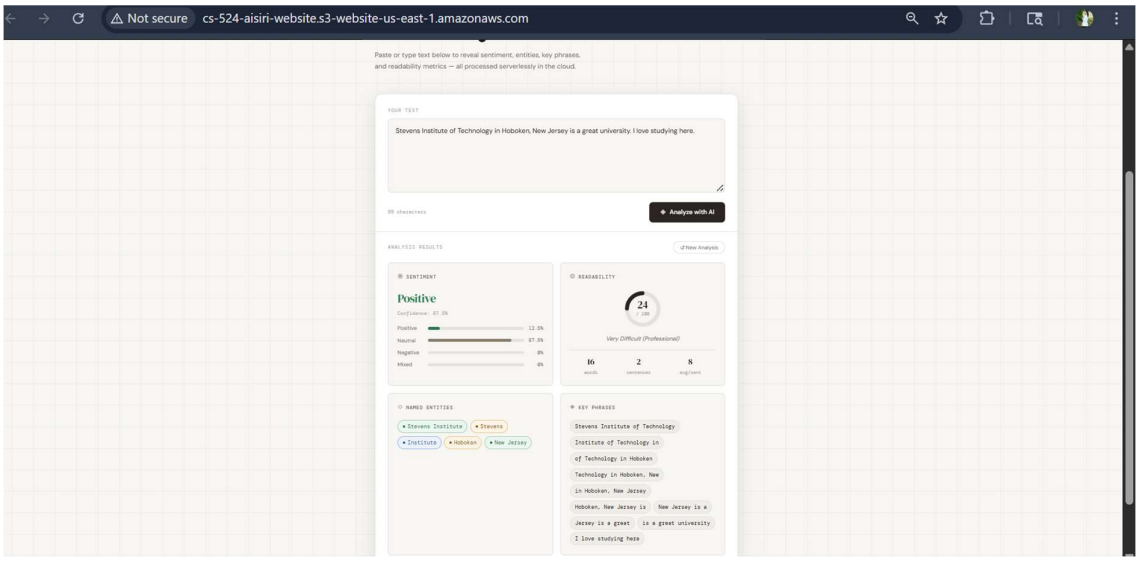
I made changes in the code because I was getting this error.

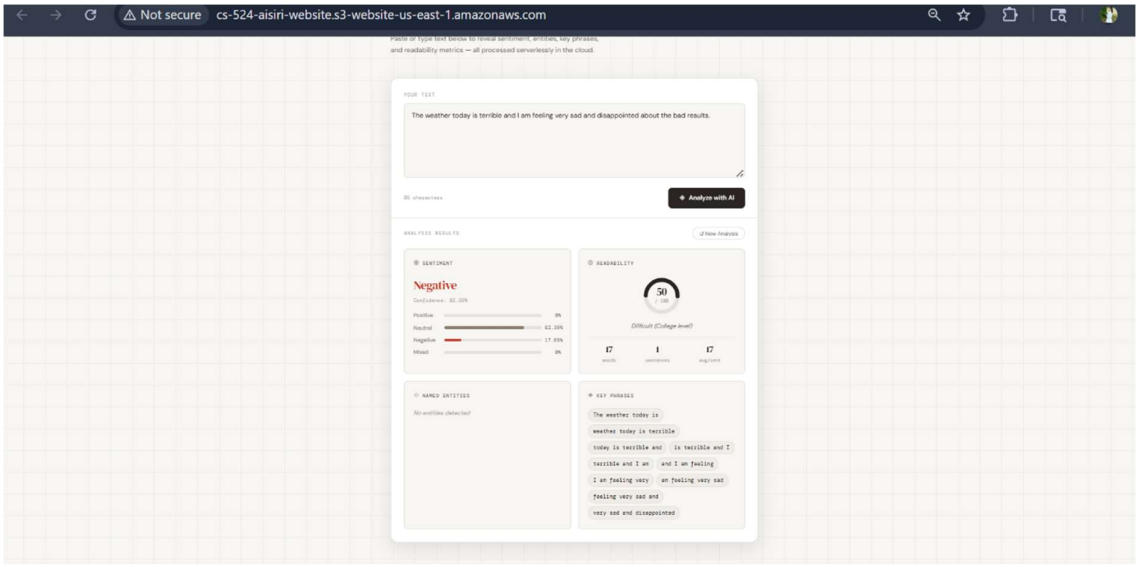


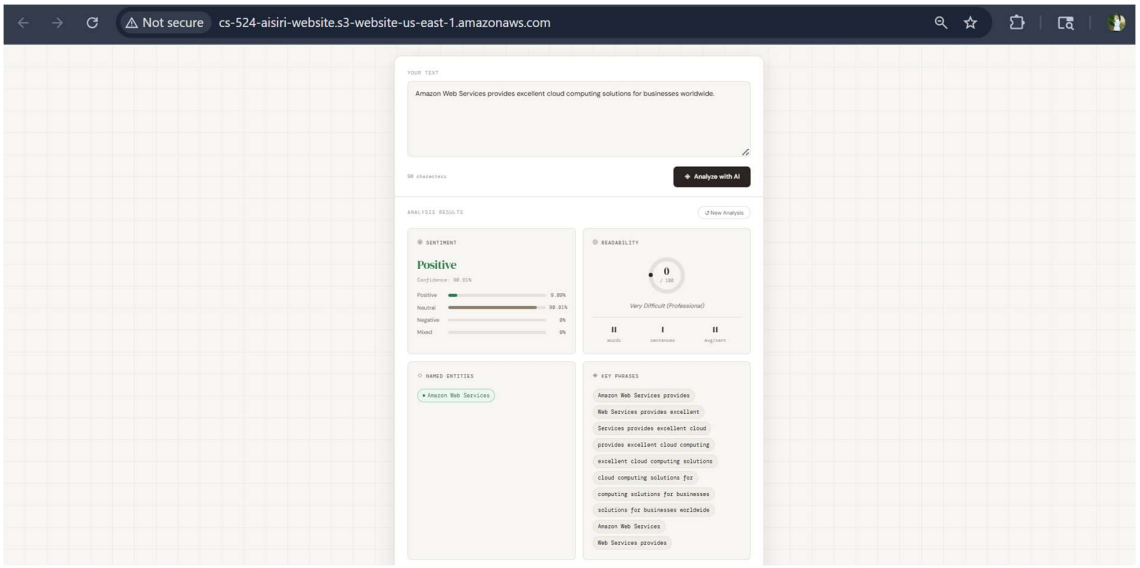


I copied the bucket website endpoint URL from the Properties tab and opened it in the browser. The website loaded correctly showing the text input and analyze button.

Step 4: Website showing the analysis results after entering sample text

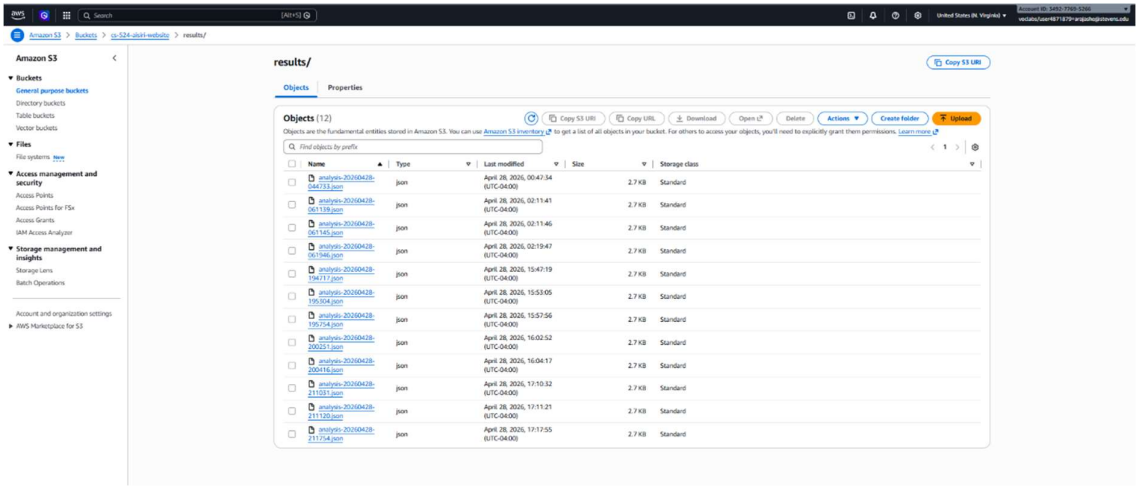






I typed in the sample sentence and clicked Analyze with AI. The results section showed up with sentiment, entities, key phrases, and readability all displaying correctly.

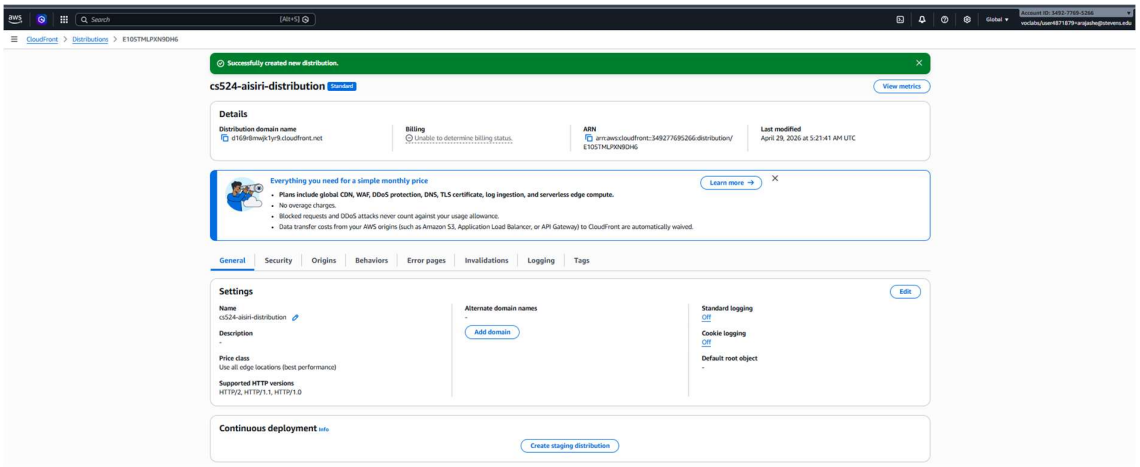
Step 5: Results folder in S3 showing JSON files saved from each analysis



I went back to the S3 bucket and found a results folder had been created automatically. Inside it there were JSON files from each analysis I had run.

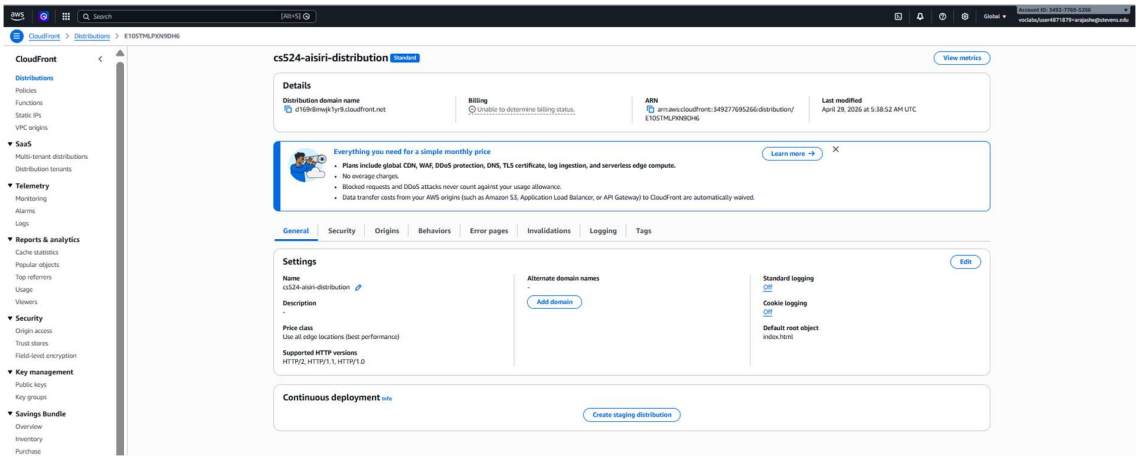
Phase 5: CloudFront HTTPS Distribution

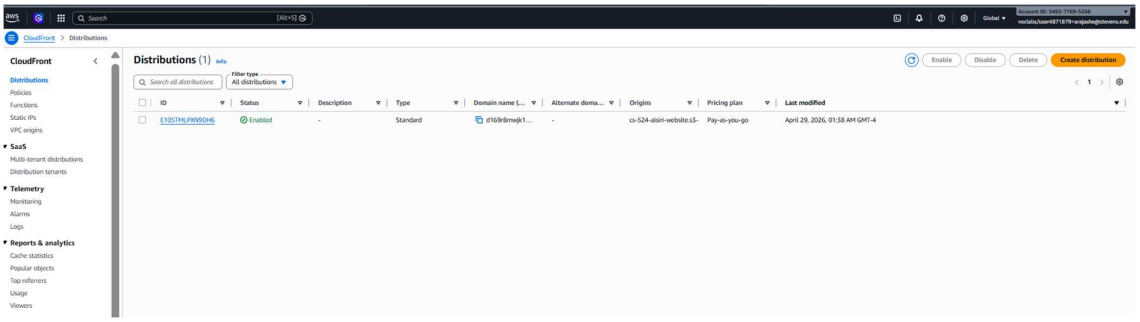
Step 1: CloudFront distribution created showing the S3 origin settings



I opened CloudFront and created a new distribution using the Pay-as-you-go plan. I selected my S3 bucket as the origin and clicked Use website endpoint when the yellow banner appeared.

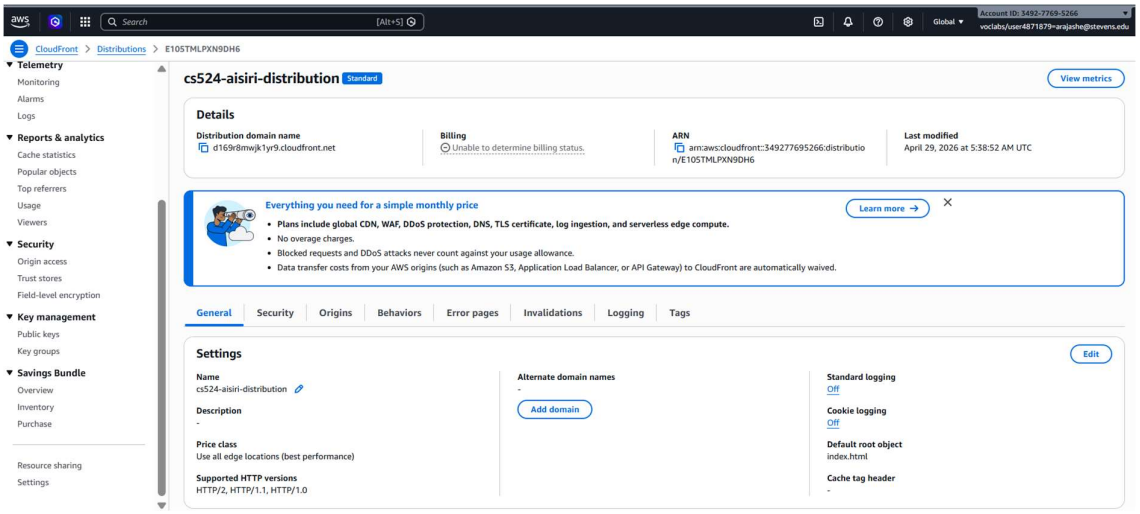
Step 2: Distribution status showing Enabled





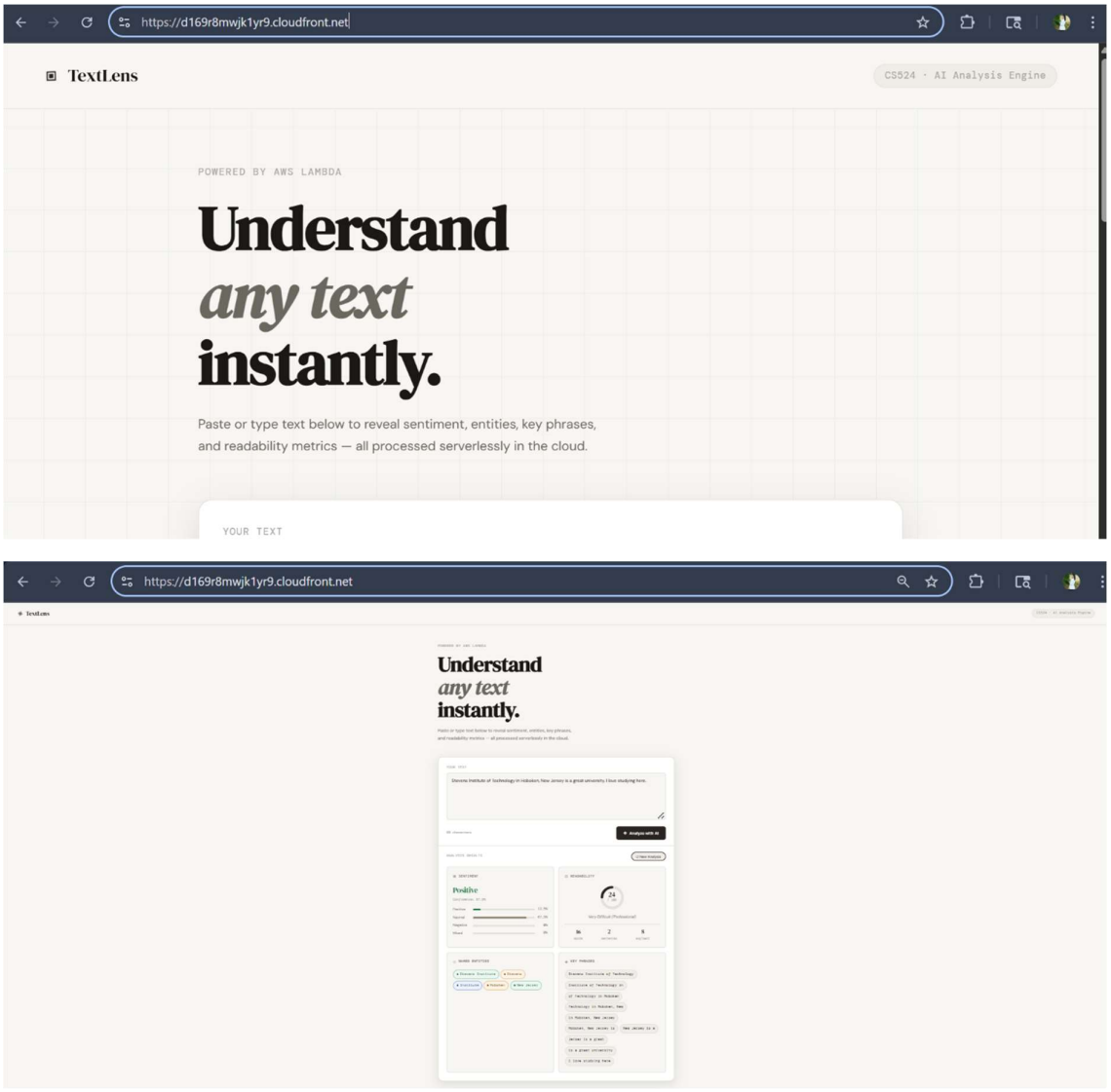
After creating the distribution I waited about 10 minutes refreshing the page. The status eventually changed from Deploying to Enabled.

Step 3: Default root object set to index.html in general settings



I clicked Edit in the General settings of the distribution and typed index.html as the default root object. This makes sure the homepage loads when someone visits the CloudFront URL.

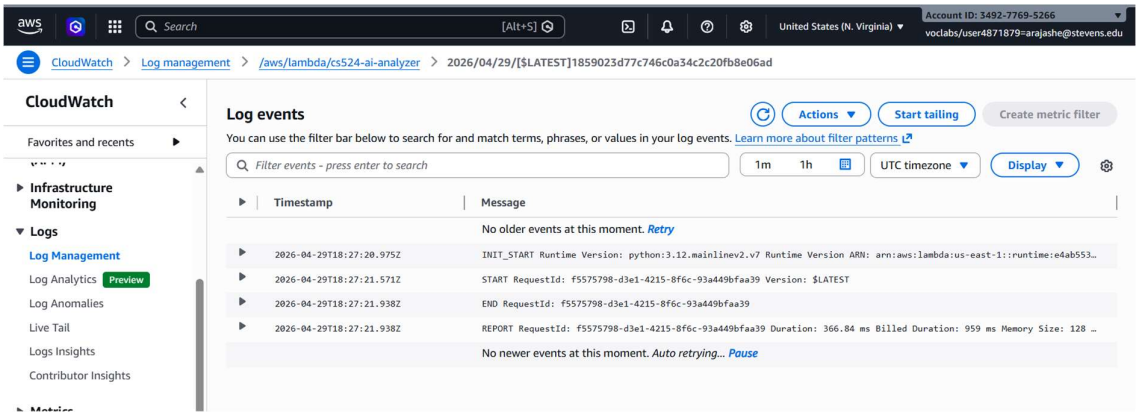
Step 4: Website loading over HTTPS using the CloudFront URL

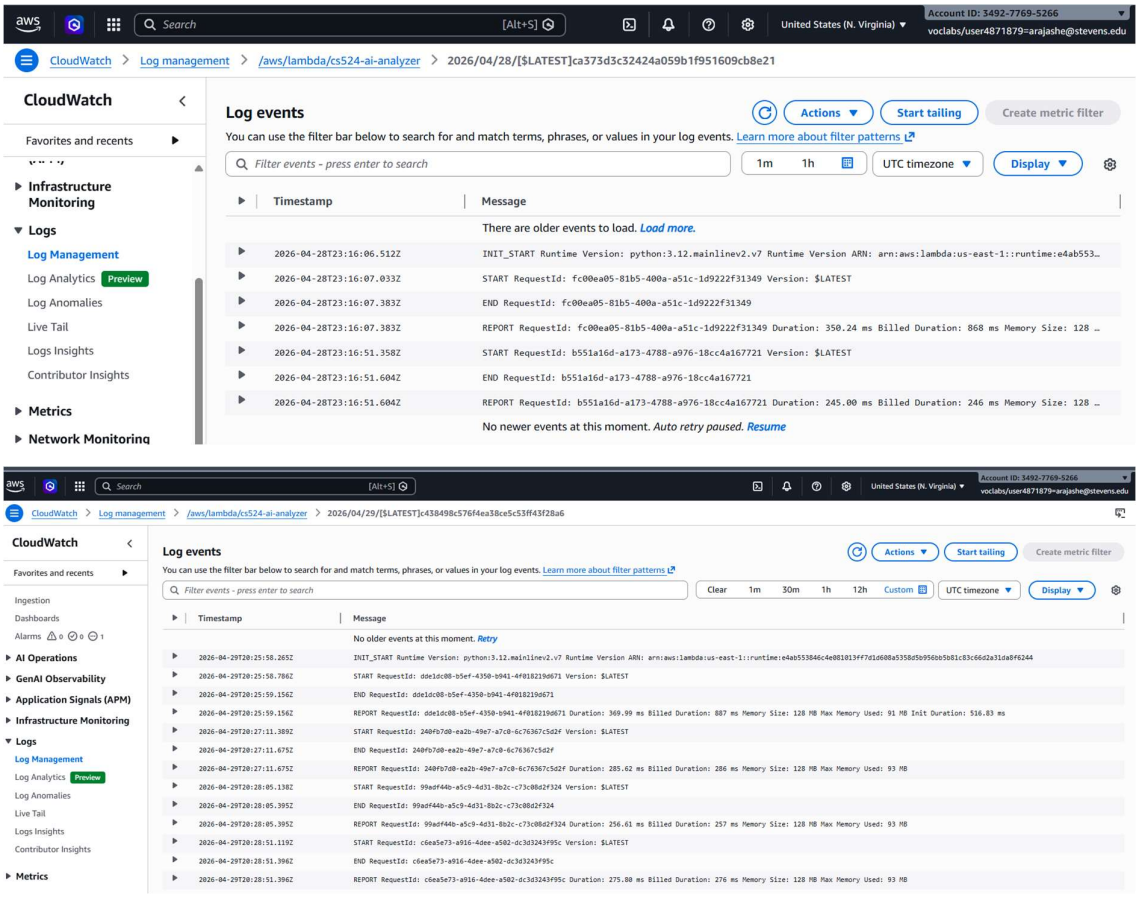


I copied the CloudFront domain name and opened it in the browser with https. The website loaded and the Analyze button worked correctly this time without any errors.

Phase 6: CloudWatch Monitoring

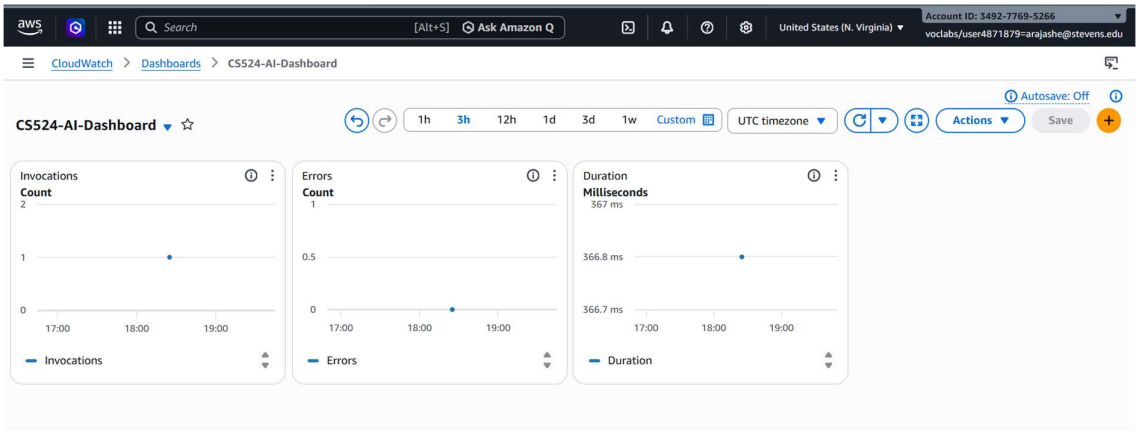
Step 1: CloudWatch log group for the Lambda function showing execution logs





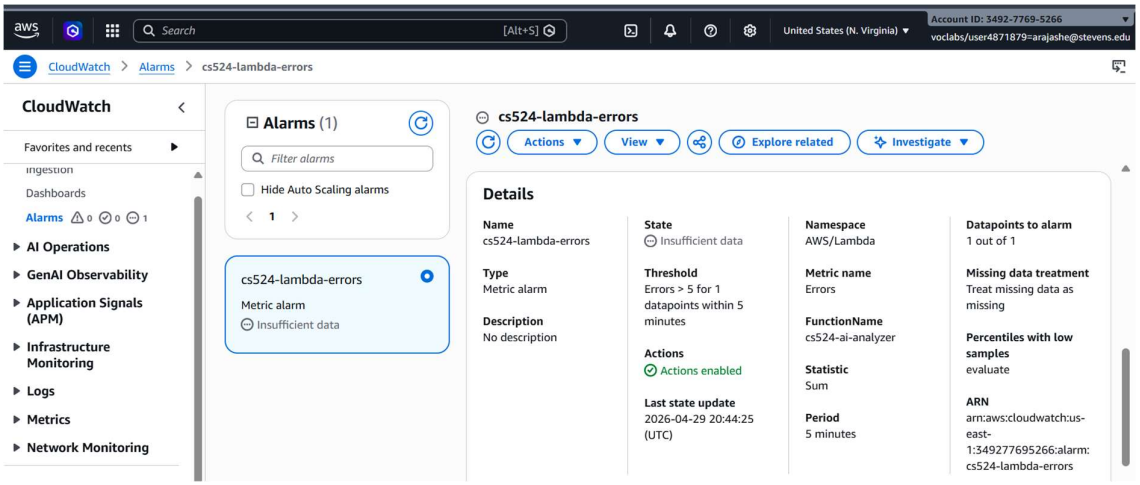
I opened CloudWatch and went to Log groups where I found the log group for my Lambda function. I clicked into one of the log streams and could see entries from each time the function ran.

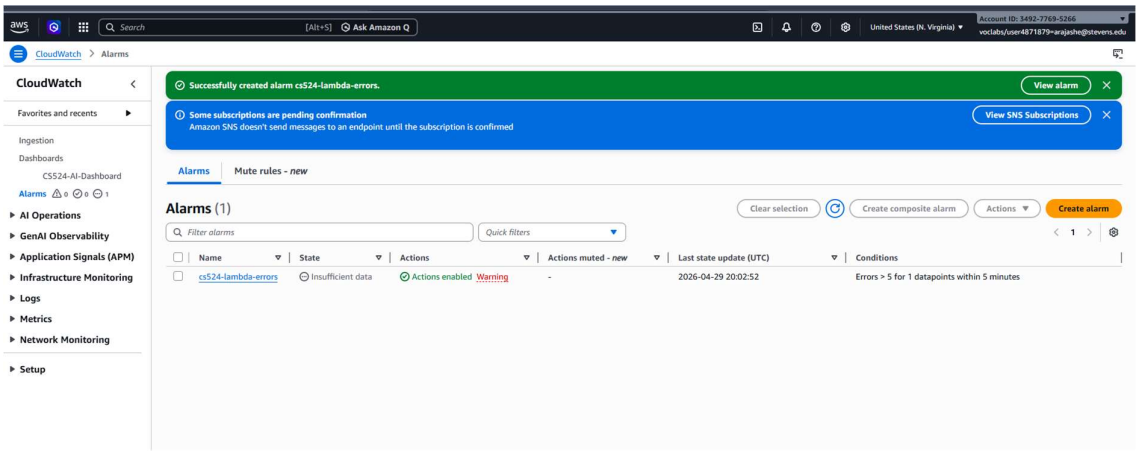
Step 2: CS524-AI-Dashboard with Invocations, Errors and Duration widgets.

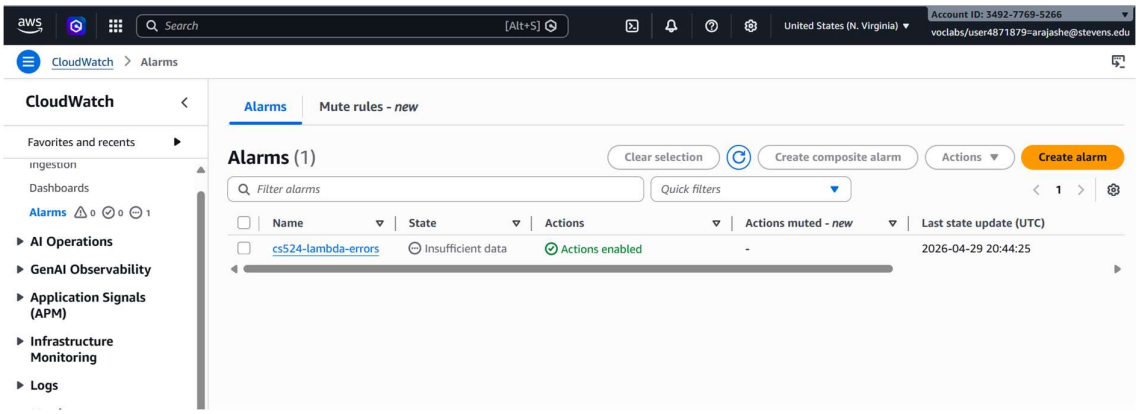


I created a new dashboard called CS524-AI-Dashboard and added three widgets for Invocations, Errors, and Duration. After running a few more analyses the widgets started showing real data.

Step 3: CloudWatch alarm created for Lambda errors

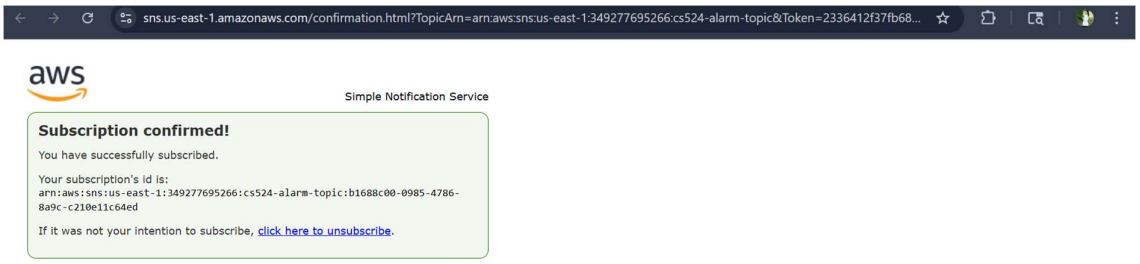






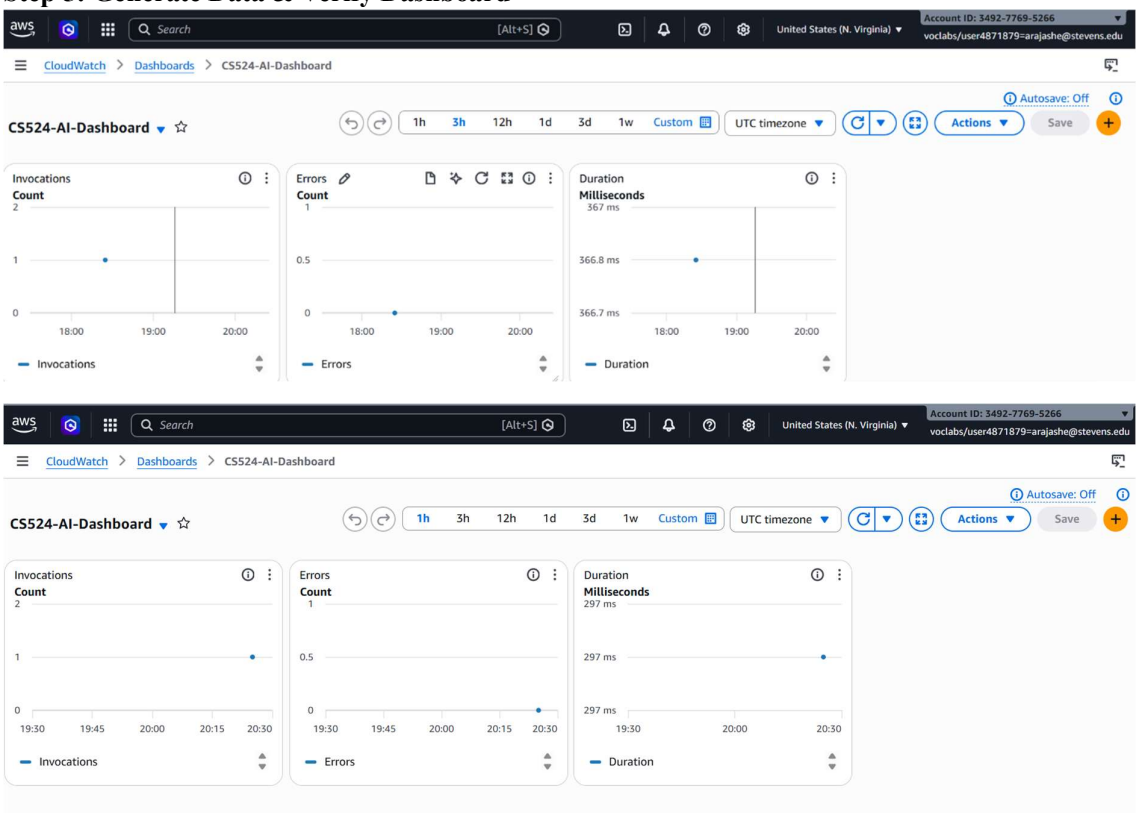
I created a CloudWatch alarm based on the Lambda Errors metric with a threshold of 5 errors in 5 minutes. I set up an SNS topic with my email address to receive notifications if the alarm triggers.

Step 4: SNS subscription confirmation email received and confirmed



I checked my Stevens email and found the AWS subscription confirmation message. I clicked the confirm link in the email and the SNS subscription status updated to confirmed.

Step 5: Generate Data & Verify Dashboard



I ran a few text analyses on the website to generate some actual usage data. Then I went back to the CloudWatch dashboard and the widgets updated showing real numbers for invocations and duration.